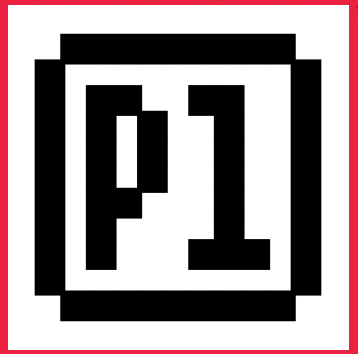


STDLIB

UNRN

Universidad Nacional
de **Río Negro**

r33

14 / 2026

malloc / free

Más stdlib de memoria

UNRN Universidad Nacional de **Río Negro**

calloc

Para crear un arreglo dinámico y en cero.

Crea un arreglo de cantidad de tamaño

```
void* calloc(size_t cantidad, size_t tamaño);
```

Un arreglo dinámico de 4 elementos

```
int* arreglo = calloc(4, sizeof(int));
```

realloc

Para cambiar el tamaño de un bloque pedido

Agregar que pasa con el puntero y los datos que
estaban antes
(el puntero puede cambiar y los datos se copian)

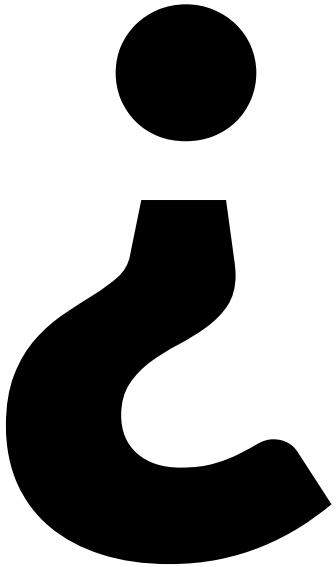
[markdown]¿Qué hace `realloc` si el puntero que se le pasa es NULL?
{
=Actúa como `malloc`, asignando un nuevo bloque de memoria.

El nuevo tamaño no debe ser cero

```
void* realloc( void* bloque, size_t new_size );
```

Pasamos de un arreglo de 4 a uno de 10

```
int* arreglo = calloc(4, sizeof(int));
arreglo = realloc(arreglo, sizeof(int) * 10);
```



Y si bloque es NULL



`realloc`, ¡puede fallar!

```
int* arreglo = calloc(4, sizeof(int));
arreglo = realloc(arreglo, sizeof(int) * 10);
```



Y dejar en `NULL` arreglo

realloc , ¡puede fallar!

```
int* arreglo = calloc(4, sizeof(int));
int* temporario = realloc(arreglo, sizeof(int) * 10);
```



Y dejar en NULL arreglo


¿Preguntas?

memset

Para inicializar memoria

Pone valor los count bytes

```
void* memset( void *destino, int valor, size_t count );
```

valor como (unsigned char)

```
char str[] = "ghghghghghghghghghgh";
puts(str);
memset(str,'a',5);
puts(str);
```

memcpy

Para copiar bloques de memoria

Retorna dest

```
void* memcpy( void *destino, void* origen, size_t cuenta );
```

Malloc y copia

```
const int origen[] = {1, 2, 3, 4, 5, 6, 7, 8};
const int largo_origen = sizeof(origen) / sizeof(int);
int *destino = (int*)malloc(largo_origen * sizeof(int));

if ((destino != NULL)){
    destino = memcpy(destino, origen, sizeof(origen));
} else {
    exit(EXIT_FAILURE);
}
```


¿Preguntas?

Argumentos main

UNRN Universidad Nacional de Río Negro

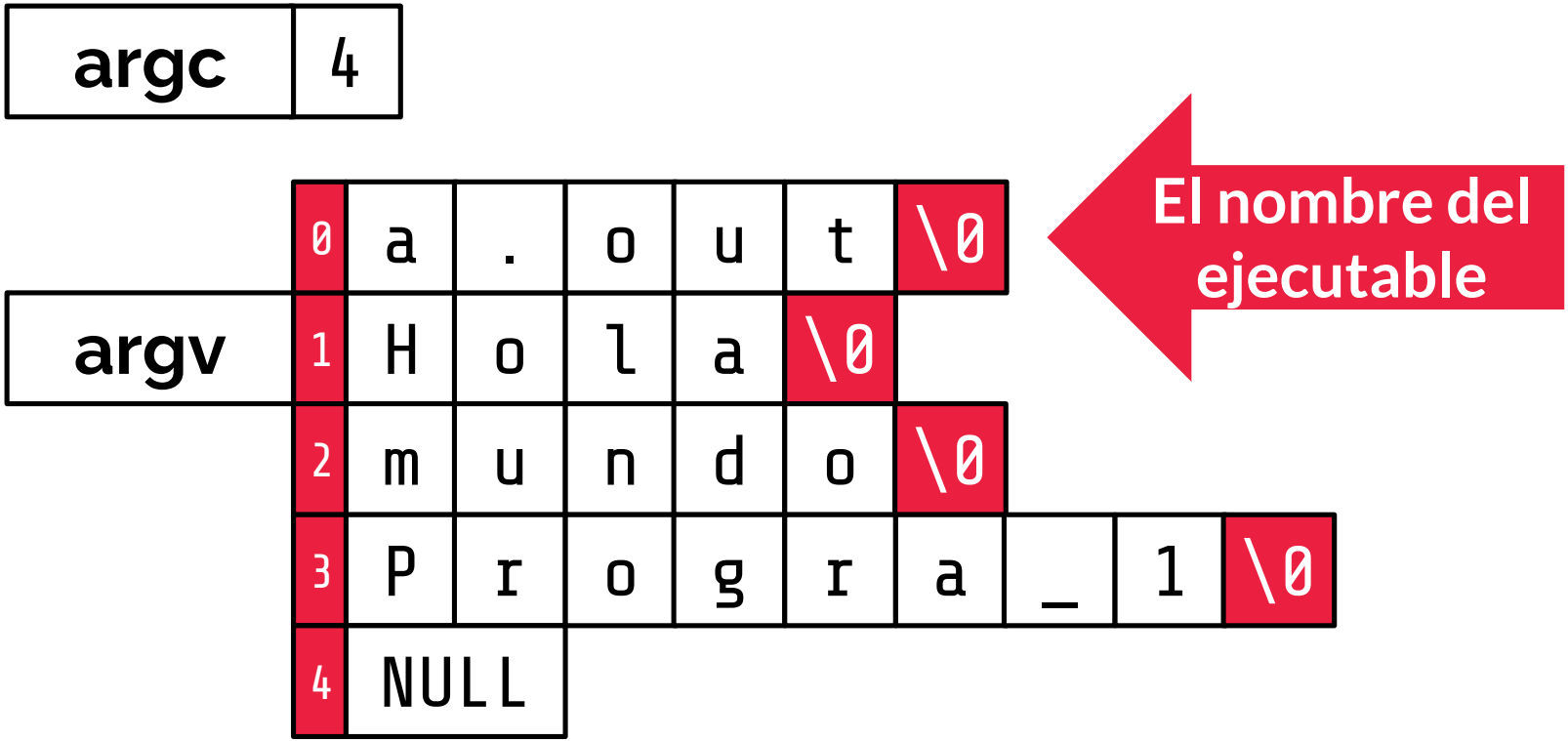
El `main` *completo* tiene esta forma

```
int main(int argc, char *argv[])
```

```
argc -> cuantos argumentos
argv -> las cadenas de argumentos
```

Siempre contiene el nombre del archivo

```
argv[0]
```



Los argumentos son un arreglo de cadenas

¡A la terminal!

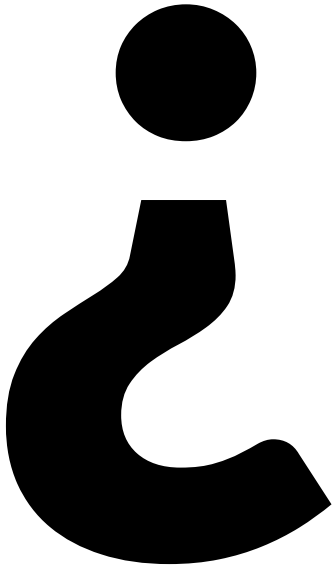


Un ejemplo completo

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 4) {
        printf("Uso: %s <r> <g> <b>\n", argv[0]);
        return 1;
    }
    const int r = atoi(argv[1]);
    const int g = atoi(argv[2]);
    const int b = atoi(argv[3]);
    printf("0x%02x%02x%02x\n", r, g, b);
    return 0;
}
```

Una indicación de que faltó[1]



Qué podemos hacer



{ string.h stdlib.h }

Convertir

```
int atoi( char *str );
```

Convierte la cadena a un int

Convertir

```
long strtol( char *str, char **str_end, int base );
```

Convierte a `int` el primer número en la cadena `str` en un número con la base indicada.

stdlib.h - conversiones de cadenas

```
atof - string a double
atoi - string a int
atol - string a long int
atoll - string a long long int
strtof - string a float
```

stdlib.h - ejecución del programa

```
abort() - cierre estrepitoso del programa
exit(int) - cierre ordenado del programa(puede ser con)
        EXIT_FAILURE por las malas
        EXIT_SUCCESS por las buenas
```

Comparar

```
int strcmp( char* izq, char* dch);
```

Compara lexicográficamente dos cadenas

Si el argumento es igual a algo que indiquemos

`string.h` - comparación

```
int strcmp(char* str1, const char* str2);
```

Compara `str1` con `str2`, indicando un ordenamiento basado en el código de los caracteres de ambas cadenas.

- <0 el primer carácter que no coincide es de valor menor
- 0 iguales
- >1 el primer carácter que no coincide es de mayor valor

Copiar

```
char* strcpy( char *dst, char *src);
```

Copia la cadena `src` en `dst`, con el terminador en el largo de `src`.

¡A la terminal!




¿Preguntas?

Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

UNRN Universidad Nacional de **Río Negro**

@unrionegro

unrn.edu.ar